

Nama : Ika Farikha

NIM : A18.2024.00168

Review Koding yang Digenerate AI

Dokumen Observasi dan Analisis Kritis Terhadap Kode yang Dihasilkan oleh AI

1. Yang Sudah Sesuai Ekspektasi

Berikut adalah aspek-aspek dari kode yang dihasilkan AI yang sudah sesuai dengan harapan dan kebutuhan tugas:

Fungsionalitas Utama

- Logika inti program berjalan dengan benar sesuai dengan spesifikasi tugas yang diberikan.
- Struktur kode terorganisir dengan baik menggunakan fungsi/modul yang terpisah, sehingga mudah dibaca.
- Penanganan input dan output sudah sesuai dengan format yang diminta pada soal.
- Algoritma yang dipilih AI relevan dan efisien untuk skala data yang dihadapi dalam tugas ini.

Kualitas Penulisan Kode

- Penamaan variabel dan fungsi cukup deskriptif sehingga kode mudah dipahami.
- AI menyertakan komentar pada bagian-bagian penting yang membantu proses pemahaman.
- Konsistensi indentasi dan gaya penulisan terjaga di seluruh bagian kode.

1.1 Contoh Kode yang Sudah Sesuai

Contoh bagian kode yang dihasilkan dan langsung bisa digunakan tanpa modifikasi:

```
# Contoh fungsi validasi input yang dihasilkan AI
def validate_input(data):
    if not data:
        raise ValueError('Input tidak boleh kosong')
    return True
```

2. Yang di Luar Ekspektasi

2.1 Di Luar Ekspektasi Secara Positif

Beberapa hal yang mengejutkan secara positif dari kode yang dihasilkan:

Kejutan Positif

- AI secara proaktif menambahkan penanganan error (try-except/try-catch) yang tidak secara eksplisit diminta, sehingga program lebih robust.
- AI menyarankan optimasi menggunakan struktur data yang lebih efisien (misalnya dictionary/hashmap) dibanding solusi naif yang mungkin terpikirkan pertama kali.
- Dokumentasi fungsi (docstring) yang dihasilkan sangat lengkap, mencakup parameter, return value, dan contoh penggunaan.
- AI memberikan alternatif pendekatan beserta perbandingan trade-off-nya, memberikan wawasan tambahan yang bermanfaat.

2.2 Di Luar Ekspektasi Secara Negatif

Beberapa hal yang tidak sesuai harapan dan memerlukan perhatian:

Hal yang Mengecewakan

- AI mengasumsikan library tertentu tersedia padahal batasan tugas melarang penggunaan library eksternal, sehingga perlu ditulis ulang dari awal.
- Kode yang dihasilkan pertama kali menggunakan gaya yang berbeda dari konvensi yang diajarkan di kelas (misalnya perbedaan gaya OOP vs prosedural).
- Untuk edge case tertentu (input sangat besar atau karakter khusus), AI tidak memperhitungkan skenario tersebut sehingga program crash saat diuji.
- AI terkadang menghasilkan kode yang 'terlalu pintar' sehingga sulit dijelaskan saat presentasi/demo.

3. Kode yang Bisa Di-improve

Berdasarkan analisis mendalam, berikut bagian-bagian kode yang memiliki potensi perbaikan lebih lanjut:

3.1 Penanganan Error yang Lebih Komprehensif

- Blok try-catch yang dihasilkan AI masih terlalu general, menangkap semua exception tanpa membedakan jenis error.
- Sebaiknya dibuat penanganan spesifik untuk setiap jenis error agar pesan yang diberikan ke pengguna lebih informatif.

Contoh perbaikan yang disarankan:

```
# Sebelum (terlalu general): try:      result = proses_data(input) except  
Exception as e:      print('Error:', e) # Sesudah (lebih spesifik): try:  
result = proses_data(input) except ValueError as e:      print('Input tidak  
valid:', e) except FileNotFoundError:      print('File tidak ditemukan, periksa  
path.')
```

3.2 Validasi Input

- Fungsi validasi yang dihasilkan hanya memeriksa satu kondisi; perlu diperluas untuk mencakup lebih banyak skenario input yang tidak valid.
- Penambahan validasi tipe data akan membuat program lebih aman dari kesalahan penggunaan.

3.3 Modularitas

- Beberapa fungsi masih menangani terlalu banyak tanggung jawab sekaligus (violating Single Responsibility Principle).
- Memecah fungsi besar menjadi fungsi-fungsi kecil yang lebih fokus akan meningkatkan keterbacaan dan kemudahan pengujian.

4. Kode yang Overengineer (Perlu Disimplifikasi)

Terdapat beberapa bagian kode yang menurut penilaian terlalu kompleks untuk kebutuhan tugas ini dan dapat disederhanakan:

4.1 Pola Desain yang Berlebihan

- AI mengimplementasikan pola Factory Pattern untuk pembuatan objek sederhana yang hanya memiliki satu implementasi. Untuk skala tugas ini, instansiasi langsung sudah cukup.
- Penggunaan Abstract Base Class (ABC) untuk hierarki kelas yang hanya memiliki satu turunan adalah berlebihan dan menambah kompleksitas tanpa manfaat nyata.

Contoh penyederhanaan:

```
# Overengineer (dihasilkan AI): from abc import ABC, abstractmethod class  
DataProcessorBase(ABC):      @abstractmethod      def process(self, data): pass  
class DataProcessor(DataProcessorBase):      def process(self, data): return  
data.strip() # Disederhanakan: def process_data(data):      return data.strip()
```

4.2 Konfigurasi yang Berlebihan

- AI membuat sistem konfigurasi berbasis file JSON untuk parameter yang nilainya tidak berubah dan tidak perlu dikonfigurasi secara eksternal dalam konteks tugas ini.
- Konstanta sederhana atau parameter fungsi sudah cukup untuk keperluan tugas.

4.3 Logging yang Tidak Perlu

- AI menambahkan sistem logging multi-level (DEBUG, INFO, WARNING, ERROR) yang sebenarnya tidak diperlukan untuk program skala kecil ini.
- Penggunaan `print()` sederhana sudah memenuhi kebutuhan debugging selama pengembangan tugas ini.

4.4 Struktur File yang Terlalu Kompleks

- AI cenderung memisahkan kode ke banyak file/modul meskipun program sebenarnya cukup ditulis dalam satu file untuk skala tugas ini.
- Struktur direktori yang disarankan AI lebih cocok untuk proyek produksi, bukan untuk tugas akademik.

5. Kesimpulan

Secara keseluruhan, AI terbukti sangat membantu dalam mempercepat proses pengerjaan tugas ini. Namun, penggunaan AI tidak dapat bersifat pasif — diperlukan pemahaman mendalam dari pemrogram itu sendiri untuk menyaring, memvalidasi, dan menyesuaikan output yang dihasilkan agar benar-benar sesuai dengan kebutuhan.

Kode yang dihasilkan AI sering kali bersifat 'production-grade' yang melebihi kebutuhan tugas akademik. Oleh karena itu, kemampuan untuk mengidentifikasi bagian mana yang relevan dan bagian mana yang berlebihan merupakan keterampilan penting yang perlu terus diasah.

Dokumen ini merupakan hasil refleksi kritis terhadap penggunaan AI dalam proses pengerjaan tugas pemrograman.